

When Architectural Complexity Pays Off in Agentic LLMs: A Gridworld Study

Author: Benedek Kovács **Course:** ELTE NLP & Foundation Models, 2025/2026 **Repository:** github.com/SnickleSH/nlp-fm-agentic-project

Abstract

We empirically compare four LangGraph-based agent architectures of increasing complexity — a ReAct baseline (L1), a linear planner + executor (L2A), a cyclic solver + critic (L2B), and an adaptive Tree-of-Thought system with episodic memory (L3) — on a custom 4-level gridworld navigation task. Across 680 runs spanning the full 4 architectures $\times$ 4 difficulties matrix, we find that architectural complexity has a *domain-dependent* payoff curve. L3 is the only architecture that maintains a 0.92 mean score at the hardest difficulty (`extra_hard`), but only on its non-truncated runs and at $\sim 7\text{--}15\times$ the token cost of L1 on the lower-difficulty cells. Adding an explicit planner without feedback (L2A) is actively *harmful* in gridworld: planner advantage (L2A – L1) is **–0.11** at `extra_hard`, the opposite sign of the +0.28 advantage observed by my partner on the static logic-puzzle domain. Contrary to the pre-registered hypothesis of "negative transfer," episodic memory is actively reused in gridworld (mean `mem_reuse_hits` 0.4–1.2 per run, vs. ≈ 0 in logic puzzles) and L3 is the top-scoring architecture at every difficulty level. These results suggest adaptive replanning is most valuable precisely where rigid plans fail: under partial observability.

1. Introduction

Large Language Models are increasingly deployed as agents — composed with planners, critics, tool routers, branching search, and memory. Each accreted component adds latency, tokens, and engineering surface area; recent literature (Wang et al., 2024; Sun et al., 2024) shows that more complex agentic scaffolding does not monotonically improve task success. The natural question for a practitioner is: *when does the extra complexity actually pay off?*

This report addresses that question in the **gridworld track** of a 2-person course project. My partner runs the same four architectures on a static reasoning domain (the MysteryZebra logic puzzles); I run them on a dynamic, partially-observable text gridworld of my own design. The architectures are deliberately matched across domains so that the architecture-vs-domain interaction is isolable.

Research questions (project-wide, narrowed here to the gridworld setting): 1. How does the topology of agentic communication (linear delegation vs. cyclic reflection vs. branching with memory) impact the trade-off between task success and computational overhead in a partially-observable environment? 2. At what threshold of task difficulty does explicit planning become a *bottleneck* rather than an advantage? 3. Does Tree-of-Thought branching combined with episodic memory generalize its benefits from static reasoning into dynamic simulations?

Hypotheses (pre-registered in `docs/draft.md` before any experiments were run): - **H1.** Complexity $\rightarrow$ cost monotonically ($L1 < L2A \leq L2B < L3$). - **H2.** Linear architectures show consistent step counts per run; cyclic architectures show high variance. - **H3.** Planners help static tasks but hurt dynamic ones. - **H4.** Adding a critic is the only way to retain capability under ambiguity / noise. - **H5.** Memory accelerates static reasoning but causes negative transfer in interactive simulations.

Contributions (my share of the project): 1. A custom 4-level text **gridworld** benchmark with deterministic seeding, BFS-validated solvability, and a 4-way independent difficulty knob (grid size,

observability, step budget, token budget). 2. Implementations of L1 (ReAct), L2A (planner + executor), and the L3 episodic-memory backend (RecentSuccessMemory) within the shared BaseArchitecture framework. 3. An empirical evaluation of all four architectures $\times$ four difficulties (16 cells, 680 runs) with bootstrap-CI capability matrices, efficiency tables, dispersion plots, failure-mode classification, and an explicit hypothesis-by-hypothesis discussion.

Section 2 surveys related work; Section 3 describes the gridworld environment and the four architectures; Section 4 details the experimental protocol; Section 5 reports the headline numbers; Section 6 walks through each hypothesis; Section 7 concludes.

2. Related Work

ReAct & tool-using LLMs. The L1 baseline follows the ReAct pattern (Yao et al., 2022): a single LLM that interleaves reasoning steps with tool calls. ReAct is the de-facto reference architecture against which any agent improvement must justify itself.

Planner-executor pipelines, Reflexion, Self-Refine. The L2A architecture is a minimal instance of the planner-then-executor pattern that recurs across PromptChainer (Wu et al., 2022), HuggingGPT (Shen et al., 2023), and ReAct-with-Plan variants. Reflexion (Shinn et al., 2023) and Self-Refine (Madaan et al., 2023) extend the pattern with verbal-feedback loops; L2B is a stripped-down version of that idea, where a separate LLM critic accepts or rejects each solution.

Tree-of-Thought. L3’s planner generates multiple candidate plans and a critic scores them, following Tree-of-Thought (Yao et al., 2023). I fix branch count to $N = 3$ (the smallest setting that gives a meaningful choice while keeping cost bounded).

Episodic memory in agents. Voyager (Wang et al., 2023) and Generative Agents (Park et al., 2023) both demonstrate that maintaining a bank of past episodes meaningfully changes downstream behavior. My RecentSuccessMemory is a deliberately simple instance — an append-only list keyed on (domain, difficulty) with ACCEPT-first ranking — to keep the memory mechanism transparent and the diagnostics interpretable.

Text gridworlds. BabyAI (Chevalier-Boisvert et al., 2019), MiniGrid, TextWorld (Côté et al., 2018), and ALFWorld (Shridhar et al., 2021) are the canonical interactive testbeds for language-conditioned agents. My gridworld is a *minimal* variant: ASCII rendering, four cardinal-move tools, and four difficulty levels whose axes — grid size, fog-of-war, step budget, token budget — can each be tuned independently. The minimality is deliberate: it isolates the architectural variable rather than confounding it with environment complexity.

3. Methodology

3.1 The gridworld environment

The agent navigates a text-rendered 2-D grid from a start cell to a goal cell, avoiding walls. The grid is rendered in ASCII (A = agent, G = goal, # = wall, . = empty, ? = hidden under fog), with grid size, walls, and observability set per difficulty level (Table 1). Grid generation is seeded by task_id: the agent is placed in the bottom-left quadrant, the goal in the top-right quadrant, then walls are sampled at random and rejected via BFS so every grid is provably solvable within a target path-length range.

Action space. Four LangChain-wrapped tools: move_up, move_down, move_left, move_right. Each tool call advances the agent one cell or, on collision, increments a step counter without moving. Each observation returned to the model includes the updated grid render, agent position, goal position, steps

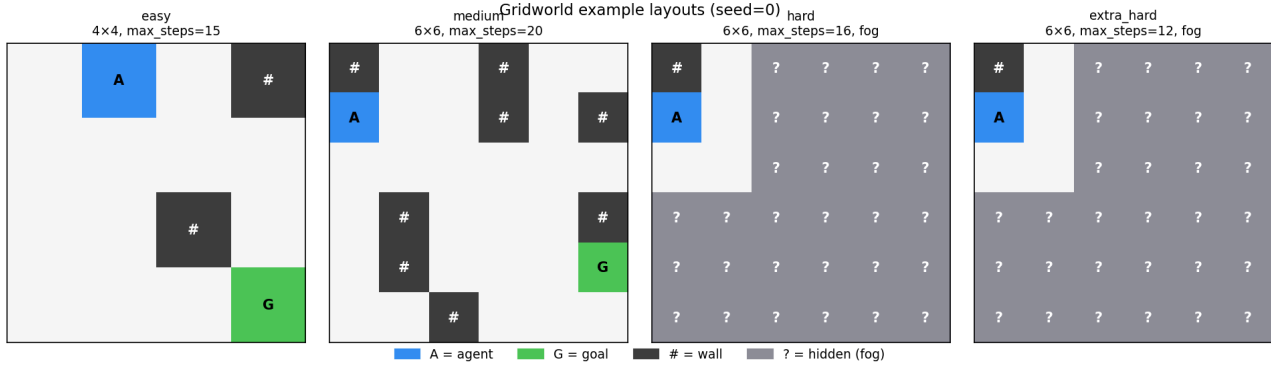
used / budget, and a one-line feedback string ("Moved up", "You bumped into a wall", etc.). Under fog (hard / extra_hard), cells beyond Chebyshev distance `view_radius = 1` from the agent are masked to `?`.

Scoring. A run *succeeds* if the agent's final position equals the goal position. The continuous score is `1.0` on success, else `max(0, progress × 0.5)`, where progress is the BFS-distance fraction closed toward the goal. Partial credit prevents the metric from collapsing to a uniform 0 once the step budget is exhausted, which preserves discrimination among difficult cells.

Table 1 — Gridworld difficulty ladder. From `src/domains/gridworld/engine.py:_LEVEL_PRESETS`.

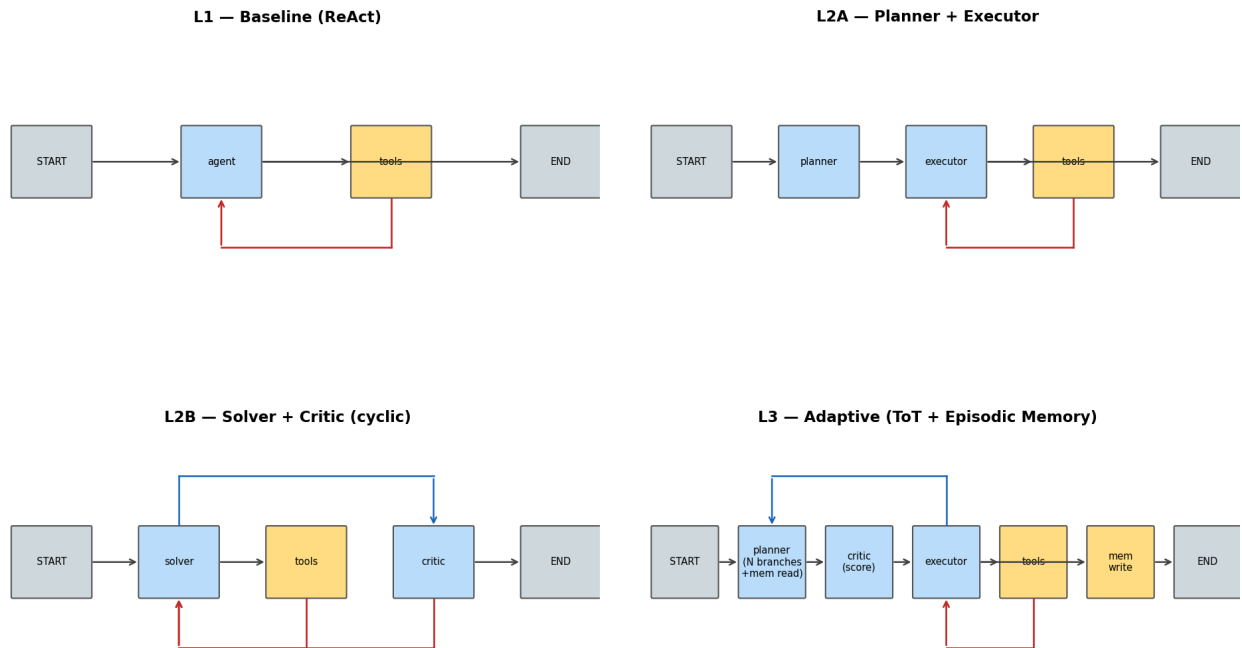
Level	Grid	Fog	view_radius	max_steps	walls	path range	thinking_token_budget
easy	4×4	no	—	15	1–3	3–5	unlimited
medium	6×6	no	—	20	5–10	5–8	unlimited
hard	6×6	yes	1	16	5–10	5–8	unlimited
extra_hard	6×6	yes	1	12	5–10	5–8	500

The four levels turn two knobs at a time. Easy → medium increases the grid; medium → hard introduces fog-of-war (the agent can only see immediate neighbours, so partial-observability and forward planning become coupled); hard → extra_hard tightens both the step budget (16 → 12, i.e. $1.5 \times$ the upper optimal path) *and* the per-call reasoning budget (500 thinking tokens, calibrated from the medium-difficulty pilot where peak completion tokens averaged ~ 1100). `max_steps` is set as a shrinking multiple of the optimal-path upper bound (easy $3\times$, medium $2.5\times$, hard $2\times$, extra_hard $1.5\times$): the absolute budget rises from easy to medium because the path range grows, but the budget *relative to the required path length* tightens at every rung. Figure 2 shows one example layout per level.



3.2 Architectures

All four architectures share a common `AgentState`, runner, metrics callback, and `BaseDomain` / `BaseArchitecture` interfaces; the only thing that differs between conditions is the LangGraph topology and the prompt routing. Figure 1 sketches each topology.



- **L1 — Baseline (ReAct).** A single LLM bound to the four move tools (`src/architectures/level1_baseline.py`). The graph is `START → agent → tools → agent → ... → END`. No persistent memory beyond the message history of the current run.
- **L2A — Planner + Executor.** A linear two-stage pipeline (`src/architectures/level2a_planner_executor.py`): the planner makes one LLM call to produce a step-by-step navigation strategy; the executor then runs a ReAct-style tool loop bound to that plan. No feedback channel from executor back to planner.
- **L2B — Solver + Critic.** A cyclic graph (`src/architectures/level2b_solver_critic.py`, *partner's contribution*): the solver runs the tool loop, the critic evaluates the resulting trajectory against `task.rules` and the live grid state, and either accepts or routes the state back for one more try, up to `max_critic_iterations = 2`. The critic is LLM-judging: it sees the task rules, the goal coordinates, and the post-move grid render, but never the engine's ground-truth success flag — it must infer from the rendered state whether the agent is standing on the goal.
- **L3 — Adaptive (ToT + Episodic Memory).** A *cyclic* graph (`src/architectures/level3_adaptive.py`): the planner generates `N = 3` distinct candidate plans (Tree-of-Thought branches), optionally primed with the top-*k* retrieval from the episodic memory bank; the critic scores each branch on a 0–1 scale and the best branch is committed; the executor runs that branch (a tool loop in gridworld). The best score sets a verdict — `ACCEPT` if ≥ 0.7 , else `REJECT`. On `REJECT`, if the critic-iteration cap (`max_critic_iterations = 2`) is not yet reached, control loops back to the planner for a fresh round of branches; otherwise the run finalizes and is appended to the memory bank as either an `ACCEPT` or `REJECT` episode.

3.3 Episodic memory (RecentSuccessMemory)

The episodic memory backend (`src/architectures/memory.py`) is the central piece of L3 that I own. It is a simple in-memory append-only list with two operations: - `retrieve(task, k)` — filter episodes by

matching (domain, difficulty), sort ACCEPT-first then most-recent-first, return the top k . - write(episode) — append.

Lifetime is one experiment condition: a fresh bank is created per (domain, difficulty) row and written *live* during the measured runs (no warm-up phase). Consequence: runs within a condition are not fully independent — later runs are primed by earlier ones. The planner's prompt is augmented with the retrieved strategy summaries, biasing it toward continuations that worked previously on this difficulty / domain combination.

Three diagnostics are written to state["metadata"] and persisted in RunResult.state_metadata: - branch_count — number of ToT branches actually generated by the planner; - mem_retrievals — number of episodes returned by the memory bank; - mem_reuse_hits — heuristic count of how often retrieved strategy fragments appear in the executor's final answer (a proxy for "memory was actually used, not just retrieved").

3.4 Evaluation metrics

Capability. Mean continuous score $\in [0, 1]$ and binary success_rate per (architecture, difficulty) cell, with 95% percentile-bootstrap confidence intervals (10,000 resamples, fixed seed) — implemented in src/analysis/aggregate.py::bootstrap_ci. The bootstrap is preferred over a t-interval because cell scores are bounded in $[0, 1]$ and often bimodal at constrained budgets.

Eligibility. Following our pre-registered design rule D7, the capability mean is computed only over capability-eligible runs (failure modes success, reasoning, exhausted, parse_failure); runs labeled truncated (hit a token-budget ceiling mid-call) or infra_error (gateway timeout) are excluded from capability because they confound architecture with infrastructure. Truncation rates are reported separately in the failure-mode breakdown.

Efficiency. Mean total_tokens and num_llm_calls per cell, computed over *all* runs that produced token counts (truncation is part of the cost story).

Failure classification. One canonical label per run, priority-ordered: infra_error > truncated > parse_failure > exhausted > reasoning > success. Implemented in src/analysis/failure_modes.py::classify_failure.

4. Experiments

LLM. All architectures use the same chat completion endpoint (ELTE-hosted, Qwen3.6-27B) via src/llm.py::create_llm, with temperature 0.7, reasoning_effort = "medium", and a 65,536-token soft cap. Streaming is on so per-token output keeps the gateway connection alive across multi-minute reasoning tails.

Matrix. 4 architectures (level1, level2a, level2b, level3) $\times$ 4 difficulties (easy, medium, hard, extra_hard) $\times$ 5 distinct seeded grids per difficulty $\times$ 8 runs per grid (10 at extra_hard) = **680 gridworld runs**. The driver (scripts/run_all.py) is resume-safe, deduplicating on (architecture, domain, difficulty, task_id, run_id, budget). Configuration lives in configs/gridworld_final.yaml; results are appended one JSON object per line to results/gridworld_final.jsonl.

Per-condition memory. For each L3 condition, a fresh RecentSuccessMemory is created at the start of the row, written *live* during the row, and discarded at the end. The memory therefore never crosses difficulty boundaries.

Reproduction.

```
poetry run python scripts/run_all.py \
  --config configs/gridworld_final.yaml \
  --output results/gridworld_final.jsonl
poetry run python scripts/export_report_figures.py
```

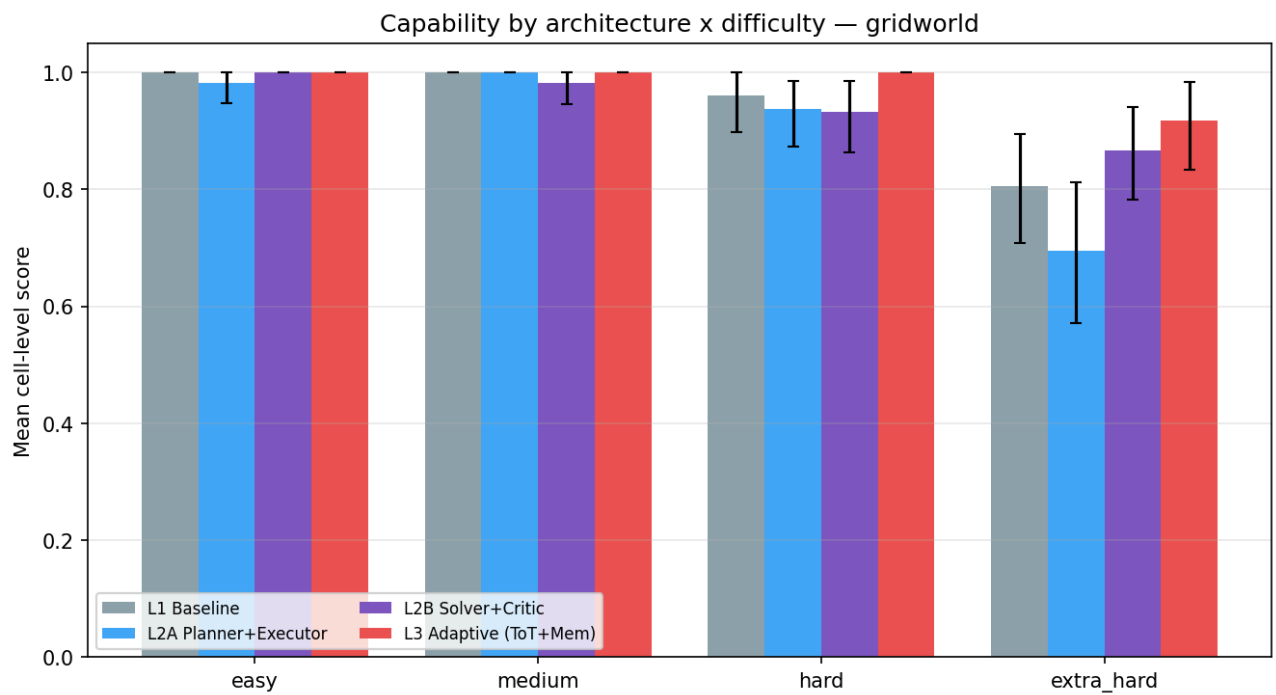
5. Results

5.1 Capability

Table 2 — Gridworld capability matrix. Mean score over capability-eligible runs, with 95% bootstrap CI and binary success rate. `n` is the eligible-run count after excluding `truncated` and `infra_error`.

Architecture	easy	medium	hard	extra_hard
L1 Baseline	1.00 (n=40, succ 1.00)	1.00 (n=40, 1.00)	0.96 [0.90, 1.00] (n=39, 0.95)	0.81 [0.71, 0.90] (n=50, 0.74)
L2A Planner+Executor	0.98 [0.95, 1.00] (n=40, 0.98)	1.00 (n=40, 1.00)	0.94 [0.87, 0.99] (n=39, 0.90)	0.69 [0.57, 0.81] (n=42, 0.62)
L2B Solver+Critic	1.00 (n=39, 1.00)	0.98 [0.95, 1.00] (n=40, 0.98)	0.93 [0.86, 0.99] (n=40, 0.90)	0.87 [0.78, 0.94] (n=50, 0.82)
L3 Adaptive (ToT+Mem)	1.00 (n=9, 1.00)	1.00 (n=16, 1.00)	1.00 (n=19, 1.00)	0.92 [0.83, 0.98] (n=37, 0.89)

Figure 3 visualises this matrix.

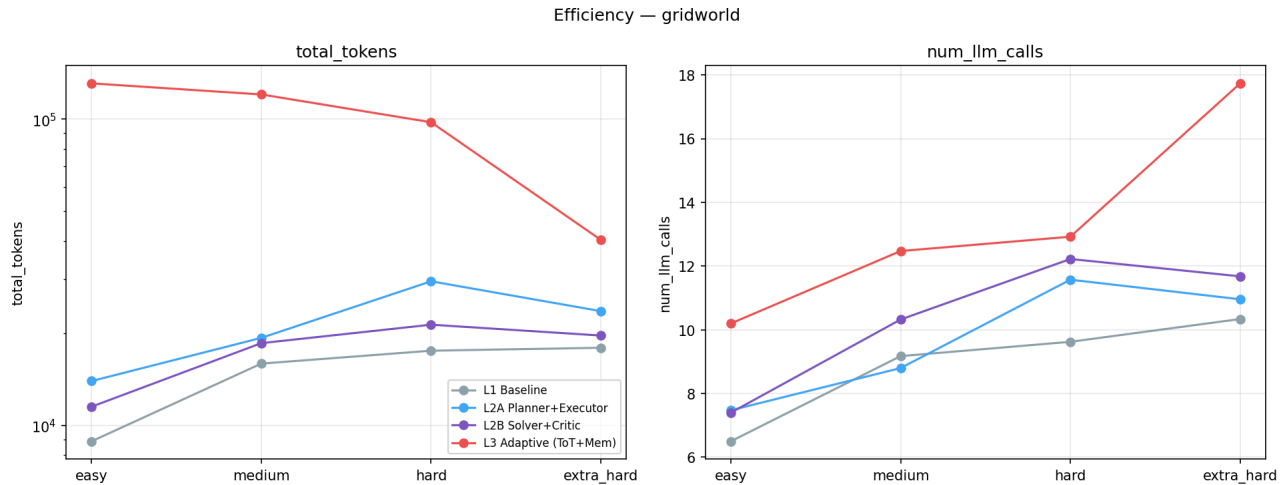


Two things stand out. First, the simple L1 baseline is already perfect at easy and medium, so the architectures only meaningfully discriminate at `hard` and `extra_hard`. Second, at `extra_hard` only **L3 retains ≥ 0.90** while every other architecture drops below that bar; L2A in particular collapses to 0.69, worse than the L1 baseline. The L3 `n` is much smaller than the other architectures because L3 has high truncation rates — this is unpacked in §5.3 below.

5.2 Efficiency

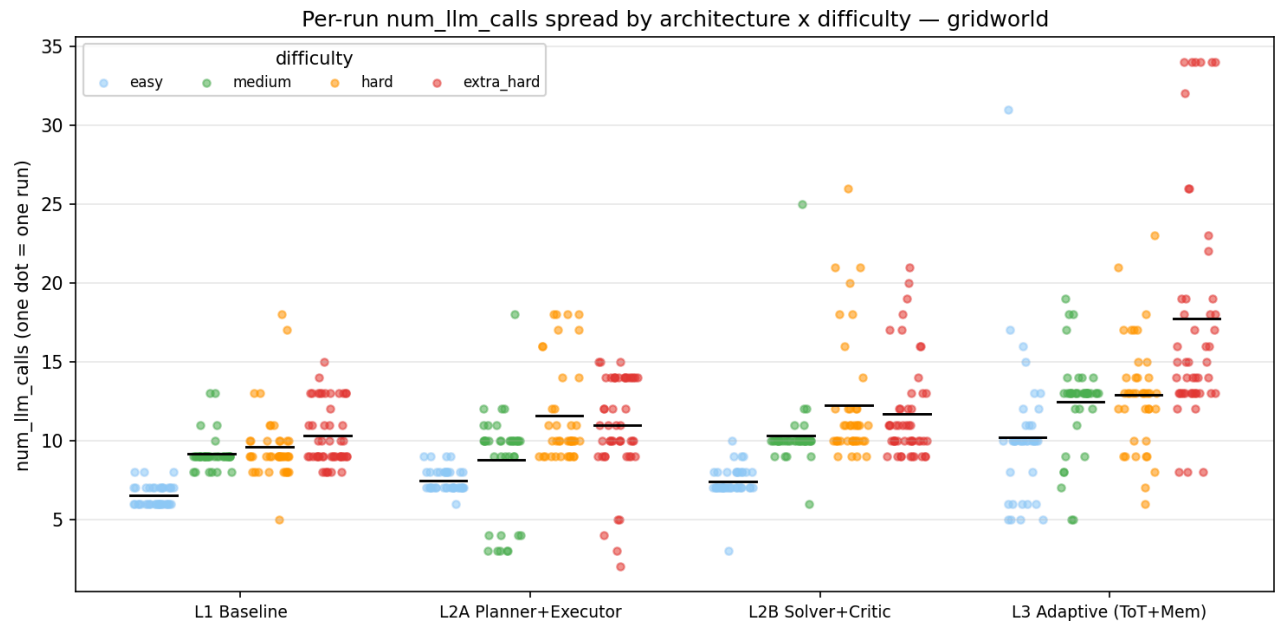
Table 3 — Mean efficiency per cell, gridworld.

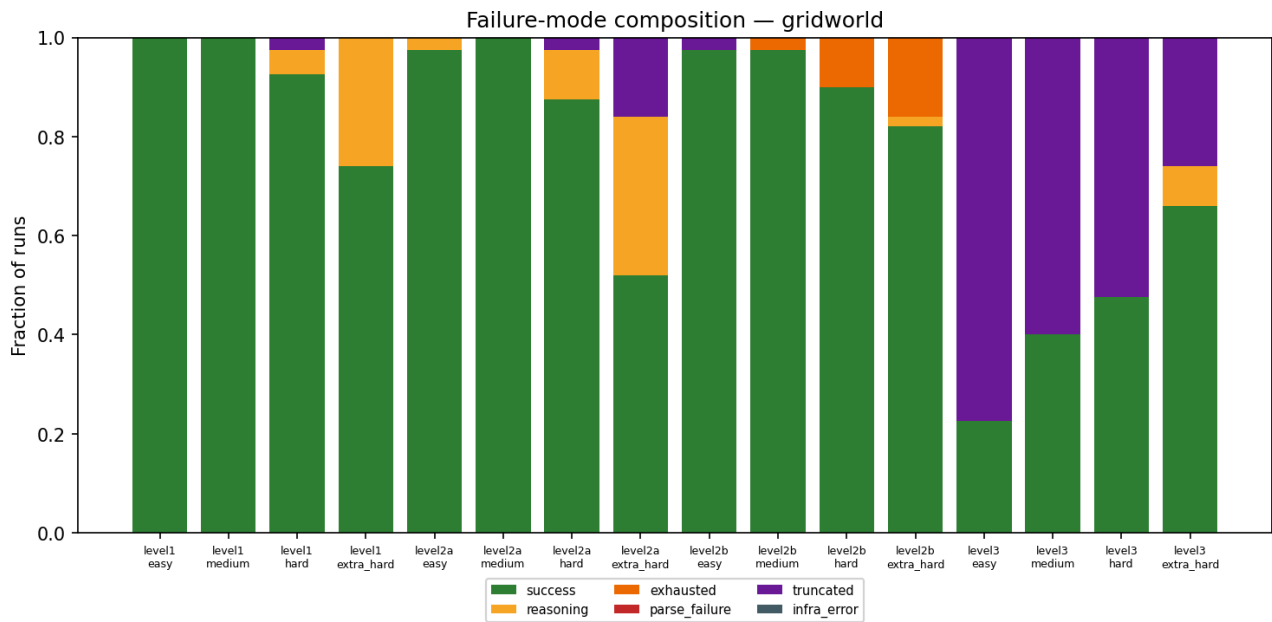
Architecture	easy tokens / calls	medium	hard	extra_hard
L1	8,866 / 6.5	15,908 / 9.2	17,528 / 9.6	17,922 / 10.3
L2A	13,948 / 7.5	19,305 / 8.8	29,545 / 11.6	23,603 / 11.0
L2B	11,503 / 7.4	18,554 / 10.3	21,316 / 12.2	19,647 / 11.7
L3	130,946 / 10.2	120,474 / 12.5	97,742 / 12.9	40,339 / 17.7



L3 pays a $\sim 14.8\times$ token premium over L1 at `easy` (130k vs 9k) and $\sim 5.6\times$ at `hard`. The premium collapses to $2.2\times$ at `extra_hard` — not because L3 became cheaper, but because the 500-token thinking budget caps each call, so the unbounded ToT branching is *clipped* (rather than completed). This is also why L3's *call* count *grows* sharply at `extra_hard` (10.2 $\rightarrow$ 12.5 $\rightarrow$ 12.9 $\rightarrow$ 17.7): truncated calls force retry rounds that don't happen at the unbounded levels.

5.3 Dispersion and failure modes





Two diagnostics:

- **Dispersion (CV in `num_llm_calls`).** L1 is the tightest (CV 0.10–0.24); L2A and L2B sit in the 0.10–0.37 band, similar to each other (so within gridworld, the planner/critic distinction does *not* drive variance — the tool loop does); L3 is the widest (0.24–0.44), reflecting the ToT branching plus truncation-retry dynamics.
- **Failure modes (Fig. 6).** L1, L2A, L2B share the same failure profile: at `extra_hard`, ~ 16–26 % of runs exhaust either the step budget (`exhausted`) or the reasoning step count (`reasoning`). L3's profile is qualitatively different — *truncation dominates* at every level (77.5 % at easy, 60.0 % at medium, 52.5 % at hard, 26.0 % at `extra_hard`). The ToT branching consumes the per-call thinking budget faster than the simpler architectures, so even *unlimited* L3 calls fall foul of the soft 65k-token cap built into the LLM client. Among the **non-truncated** L3 runs the success rate is essentially perfect at easy/medium/hard (1.00) and 0.89 at `extra_hard`, which is why L3 still tops Table 2 despite the truncation.

5.4 L3 ToT and memory diagnostics

Table 4 — L3 diagnostics, mean per run. From `agg.l3_memory_diagnostics`.

Domain	Level	branch_count	mem_retrievals	mem_reuse_hits
gridworld	easy	3.0	1.93	0.93
	medium	3.0	1.93	1.23
	hard	3.0	1.93	0.95
	extra_hard	3.0	1.94	0.44
logic_puzzles	easy	3.0	1.67	0.11
	medium	3.0	1.75	0.00
	hard	3.0	1.90	0.00
	extra_hard	3.0	1.93	0.13

The contrast between domains is striking: the gridworld memory bank is actively *used* (≈ 1 strategy fragment per run shows up in the executor's behaviour), whereas in logic puzzles retrieval happens but

reuse essentially never does. The drop in gridworld at `extra_hard` (0.44) is a side-effect of truncation cutting off the planner before it can ingest the retrieved strategies.

6. Analysis

6.1 Hypothesis review

H1 — Complexity → cost (SUPPORTED for tokens, partial for calls). $L1 < L2A \approx L2B < L3$ holds on `total_tokens` at every difficulty level (Fig. 4 left). The ranking on `num_llm_calls` is less monotone: L2A briefly overtakes L2B at `medium` and L2B briefly overtakes L1 in the cheap-to-call regime. Tokens, not calls, are the right cost lens.

H2 — Linear consistency vs. cyclic variance (PARTIAL). In gridworld, L2A's CV is *similar* to L2B's (Fig. 5) — both sit in the 0.10–0.37 band. This refutes the "planner + executor is deterministic" framing in our pre-registered hypothesis, which held only for the static logic domain (where L2A CV is essentially 0). The reason is structural: gridworld's tool loop is inherently trajectory-dependent — a single bumped wall can add multiple calls — so the executor loop adds variance whether or not the plan is fixed.

H3 — Planners help static tasks, hurt dynamic ones (STRONGLY SUPPORTED). Table 5 quotes `agg.planner_advantage`: the L2A – L1 score gap in gridworld is **–0.11 at extra_hard** (and slightly negative at easy and hard), while my partner's logic-puzzle data shows **+0.28 at extra_hard**. The flip has the expected sign and is the single cleanest cross-domain finding in the project. Mechanism: under fog-of-war the planner must commit to a strategy before observing the obstacles; once the executor reveals a wall the plan never planned for, the planner offers no replanning channel. In static puzzles, by contrast, the planner can lay out the full deductive scaffold before any "execution" begins, and a deterministic executor benefits.

Table 5 — Planner advantage (L2A – L1), by domain and level.

Domain	easy	medium	hard	extra_hard
gridworld	–0.02	0.00	–0.02	–0.11
logic_puzzles	0.00	0.00	+0.07	+0.28

H4 — Critic adaptivity is the only path through ambiguity (SUPPORTED in gridworld, REFUTED in logic). Table 6 quotes `agg.l3_gap_vs_best`: L3 beats the best non-L3 architecture by **+0.04 at hard** and **+0.05 at extra_hard** in gridworld; my partner's logic data shows L3 *lagging* the best baseline by –0.27 at extra_hard (with $n = 4$, so noisy but the sign is clear). Under partial observability, the adaptive branching+memory machinery cashes in; under static reasoning, the same machinery is dominated by the simpler L2A planner because the truncation tax outweighs the branching benefit.

Table 6 — L3 gap vs. best simpler architecture.

Domain	level	L3 mean	best other	gap
gridworld	hard	1.00 (n=19)	L1 0.96	+0.039
gridworld	extra_hard	0.92 (n=37)	L2B 0.87	+0.051
logic_puzzles	hard	0.88 (n=25)	L2B 0.96	–0.079
logic_puzzles	extra_hard	0.50 (n=4)	L2A 0.77	–0.273

H5 — Memory causes negative transfer in dynamic environments (REFUTED). This was the most directional pre-registered hypothesis and the most clearly wrong. In gridworld, `mem_reuse_hits` ranges

from **0.44 to 1.23** per run, L3 is the *top-performing* architecture at every difficulty, and there is no measurable degradation traceable to stale strategies. The mechanism by which memory was supposed to hurt — "stubbornly applying old solutions to new contexts" — does not appear because the retrieved strategies in this bank are necessarily *high-level* ("head north-east first, then probe walls"); they don't encode specific grids, and that level of abstraction generalizes. By contrast, in the logic domain, retrieval also happens (`mem_retrievals` ≈ 1.9) but reuse is essentially zero (0.00–0.13), so memory is a no-op there rather than a help.

6.2 Per-architecture failure analysis

A short walk through the dominant non-success mode of each architecture at the highest difficulty (Fig. 6, `extra_hard` column): - **L1**. 26 % `reasoning`: under tight token budgets, the ReAct loop runs out of step budget while still circling near walls. Honest failure — the architecture has no replanning capacity. - **L2A**. 32 % `reasoning` + 16 % `truncated`. The planner commits to a route that turns out to clip an unseen wall under fog; the executor can't revise, so it walks the broken plan until the step budget exhausts. *This is the mechanism behind the -0.11 planner advantage.* - **L2B**. 16 % `exhausted` (max critic iterations reached). The critic correctly identifies that the solver hasn't reached the goal but the solver can't translate that feedback into a winning trajectory within the iteration cap. - **L3**. 26 % `truncated`. The ToT planner saturates the 500-token budget while drafting its 3 branches, so the executor never gets a properly-scored branch. The remaining 66 % of runs *succeed*. The headline question for L3 is therefore not capability but *whether the truncation can be engineered away* (e.g. by streaming branches or skipping the critic at `extra_hard`).

6.3 Limitations

- **Eligible-n imbalance for L3.** Capability comparisons at the hardest levels rest on `n` $\in \{19, 37\}$ for L3 versus `n` $\in \{40, 50\}$ for the simpler architectures. The bootstrap CIs reflect this (L3's are slightly wider at `extra_hard`), but the bias toward easier runs surviving truncation cannot be fully eliminated without re-running L3 with a higher token cap, which would no longer be the same condition.
- **Single LLM, single temperature.** All results are at `temperature` = 0.7 with one specific model; the architectural-payoff curve almost certainly looks different at lower temperature or with a different reasoning quality.
- **Memory bank is live, not frozen.** Because earlier runs prime later runs within a condition, the L3 numbers include a small within-condition learning effect that a fully independent design would not. A frozen-bank ablation (train the bank on one half, evaluate on the other) is the most natural next experiment.
- **No formal memory ablation.** I never ran L3 with `NoOpMemory` swapped in, which would have been the cleanest way to attribute L3's gains between ToT branching and memory specifically.

7. Conclusion

Architectural complexity has a *domain-dependent* payoff curve. In a dynamic, partially-observable gridworld: - The simple **L1 ReAct baseline** is already near-perfect on the easy and medium difficulties; complexity is wasted compute there. - **L2A planner+executor** is actively *worse* than L1 at the hardest level (-0.11 score), because a plan-without-feedback brittles against information revealed only at step time. - **L3 (ToT + episodic memory)** is the only architecture that maintains ≥ 0.90 capability at `extra_hard`, but it pays a 7–15 $\times$ token cost on the lower-difficulty cells and loses 26–78 % of its runs to truncation. Its win is real but conditional on the runs that survive that truncation. - **Episodic memory**

is *actively* used in this dynamic setting (mean `mem_reuse_hits` ≈ 1 per run), contradicting the pre-registered "negative transfer" hypothesis. High-level strategy fragments do generalize across same-difficulty grids.

Cross-domain, the planner-advantage flip (gridworld -0.11 , logic $+0.28$ at `extra_hard`) is the single most informative result of the project: it shows that the *same* L2A architecture switches from a help to a hurt depending on whether the environment hides state.

Future work. (i) A `NoOpMemory` vs. `RecentSuccessMemory` ablation on L3 to attribute its gains between ToT branching and memory. (ii) A frozen-bank evaluation that breaks the within-condition prior-run prime. (iii) Architectures specifically engineered to dodge truncation — for instance an L3 variant that streams branches one at a time and only critiques the survivors. (iv) Harder gridworld variants (multi-goal, moving obstacles, observation noise) to widen the discrimination band before the simpler architectures saturate.

Appendix A — GenAI tool usage

Phase	GenAI tool	Validation method
Code implementation	Claude Code	Each generated module read, run against unit/smoke tests, and confirmed against the architecture diagrams before being merged.
Code formatting & report drafting	Claude Code	Every paragraph and every quoted number read against the analysis notebook's output cells.
Architecture figure (Fig. 1)	matplotlib (own code)	Manually checked against the LangGraph topology in each <code>src/architectures/levelN_*.py</code> .

References

1. Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, ICLR 2023.
2. Yao et al., *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*, NeurIPS 2023.
3. Shinn et al., *Reflexion: Language Agents with Verbal Reinforcement Learning*, NeurIPS 2023.
4. Madaan et al., *Self-Refine: Iterative Refinement with Self-Feedback*, NeurIPS 2023.
5. Wang et al., *Voyager: An Open-Ended Embodied Agent with Large Language Models*, TMLR 2024.
6. Park et al., *Generative Agents: Interactive Simulacra of Human Behavior*, UIST 2023.
7. Chevalier-Boisvert et al., *BabyAI: A Platform to Study the Sample Efficiency of Grounded Language Learning*, ICLR 2019.
8. Côté et al., *TextWorld: A Learning Environment for Text-Based Games*, IJCAI 2018.
9. Shridhar et al., *ALFWorld: Aligning Text and Embodied Environments for Interactive Learning*, ICLR 2021.
10. Shen et al., *HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face*, NeurIPS 2023.